QualiAdept

Session 7 - Static typing

Chapter 1: Introduction to TypeScript - Why did JavaScript evolve? 3
Learning Objectives 3
1.1 What is TypeScript (TS)? 3
1.2 What does "Typing" actually mean? 4
1.3 Brief History: Why Microsoft and why now? 4
1.4 Dynamic Typing (JS) vs. Static Typing (TS): Practical Examples 5
1.5 Why does Playwright use TypeScript? 6
1.6 💡 Did you know...? 7
1.7 🧠 Interview Trap Question (Interview Challenge) 7
1.8 🤖 Engineer Mindset in the AI Era: The "Hallucination-Free" Context 8
Chapter 2: First lines of code - Primitive types, any, unknown, and void 9
Learning Objectives 9
2.1 Where do we run the code? Playwright Magic vs. Browser Reality 9
2.2 Primitive Type Annotations 10
TypeScript Magic: Type Inference 11
2.3 The "Bad Boy" of TypeScript: The any Type 11
2.4 The Smart Alternative: The unknown Type 12
2.5 Functions and the void Type (Nothing) 13
2.6 🧠 Interview Challenge (Technical Question) 13
2.7 🤖 Engineer Mindset in the AI Era: The "Laziness" of Generative Assistants 14
Chapter 3: Advanced Structures - Interfaces and Custom Types 15
Learning Objectives 15
3.1 The Problem: Chaotic JavaScript Objects 15
3.2 Custom Types (Type Aliases) and Union Types 16
3.3 The Gold Standard for Data: Interfaces 16
3.4 Interface Magic: ? (Optional) and readonly 17
3.5 How do we use Interfaces in Playwright Automation? 18
3.6 🧠 Interview Challenge (Technical Question) 18
3.7 🤖 Engineer Mindset in the AI Era: "Contract-Based Prompting" 19
Chapter 4: The Command Center - Configuring tsconfig.json 20
Learning Objectives 20

4.1 Why was tsconfig.json needed? (Historical Context) 20
4.2 Interactive Lab: How flags change your code....21
Mission 1: target (The Time Machine)....21
Mission 2: noImplicitAny (The Laziness Detector) 21
Mission 3: strictNullChecks (The Absolute Savior in QA Automation)....22
The Supreme Rule: "strict": true 23
4.3 Minimalist tsconfig.json example for Playwright 23
4.4 💡 Did you know...? 23
4.5 🧠 Interview Challenge (Technical Question)....24
4.6 🤖 Engineer Mindset in the AI Era 24
Chapter 5: Advanced Concepts (Bonus) - Type Assertions (Type Casting) 26
Learning Objectives....26
5.1 The Problem: When TypeScript knows less than you do....26
5.2 The Rescue: The as Keyword (Type Casting) 27
5.3 The Hidden Danger: Lying to the Compiler 28
5.4 Connection to Playwright: Executing code in the Browser....28
5.5 🤖 Engineer Mindset in the AI Era: Blind Assertions 29
Chapter 6: Session 7 Assignment & Extra Practice (Towards Senior Level) 29
6.1 Homework Assignment (Session 7 Project) & QualiAdept Cloud Evaluation Platform 30
🏢 The Business Scenario (The Business Task): 30
📋 Technical Requirements (Acceptance Criteria for the validation platform): ...30
💡 QA Automation Analogy (Why this is vital for Playwright): 31
6.2 Bonus Challenge (Optional): Absolute safety with unknown....32
6.3 🔑 Solution Keys (Answers & Explanations) 32
Complete Session 7 - TypeScript Core 34
📍 Exercise 1: Functions, Primitives, and Type Inference 35
📍 Exercise 2: Union Types & Literal Types (Restricted Options) 35
📍 Exercise 3: Safety with unknown (Type Guards) 36
📍 Exercise 4: Interfaces and Arrays (Data Manipulation) 37
📍 Exercise 5: Classes and Encapsulation (Page Object Model) 38
📍 Exercise 6: Inheritance (Extends & Super) 39
📍 Exercise 7: DOM Manipulation and Type Casting (as)....40

Chapter 1: Introduction to TypeScript - Why did JavaScript evolve?

Learning Objectives

- 🎯 Understand the concept of "Typing" and the fundamental difference between a dynamically typed language (JavaScript) and a statically typed one (TypeScript).
- 🎯 Learn the story of TypeScript's creation and the reasons why the tech giant Microsoft built it.
- 🎯 Successfully answer technical interview scenarios regarding TypeScript execution in the browser.
- 🎯 Discover why Playwright and the entire Enterprise Software industry have adopted TypeScript as the absolute standard.

1.1 What is TypeScript (TS)?

In previous modules, you learned **JavaScript (JS)**. This is the native language of web browsers, incredibly flexible, but sometimes... *too flexible*. This flexibility frequently leads to bugs that only appear when the user clicks a button (at *runtime*).

TypeScript is NOT a completely new language. It is a **"Superset"** of JavaScript, created and maintained by Microsoft.

Think of TypeScript as *JavaScript with strict rules, an automatic spell-checker, and an advanced error-radar system*.

- **The Golden Rule:** Any valid JavaScript code is automatically valid TypeScript code. But TypeScript adds layers of safety on top of it.
- **How does it run?** Browsers (Chrome, Firefox) and Node.js **DO NOT** know how to read TypeScript. Therefore, TS code must be translated (compiled/transpiled) back into clean JavaScript before it is run. Playwright does this translation for you automatically in the

background!

1.2 What does "Typing" actually mean?

To understand why TypeScript was invented, we first need to define the concept of "Typing".

Typing represents the set of rules by which a programming language assigns a category (a "data type" like String, Number, Boolean) to your variables, and how strictly it enforces these rules.

- **Dynamic Typing (The JavaScript approach):** The computer "guesses" the data type on the fly, only when the code is actually running (at *runtime*). A box (let) can hold a number on one line of code, and 5 lines later receive text, without the language complaining. It is a fast system for building prototypes, but dangerous when scaling up.
- **Static Typing (The TypeScript / Java / C# approach):** You, as the programmer, declare the data type *in advance*, at the moment you write the code. If you declare that a box holds numbers, it becomes strict. If you try to put text in it, your editor will block your script (at *compile-time*) and throw an error before you even get to run the test.

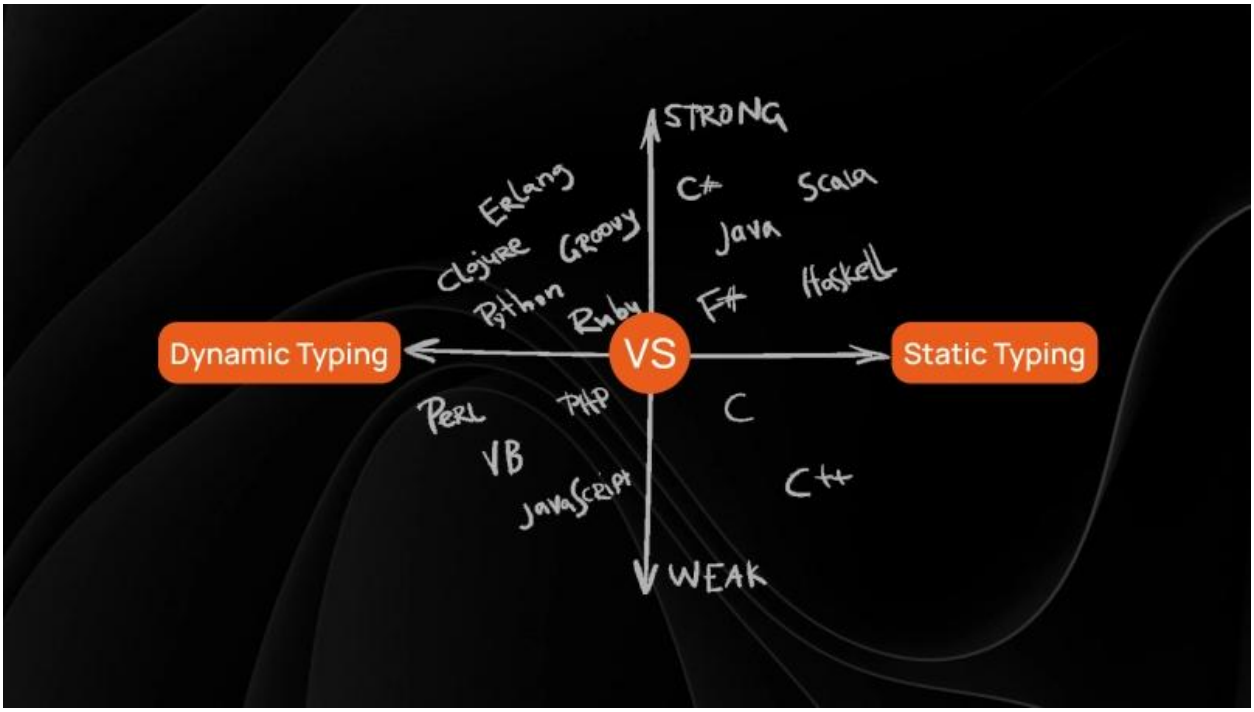


Fig. 1 - Dynamic Typing vs Static Typing

1.3 Brief History: Why Microsoft and why now?

📝 The JavaScript language was created in 1995 in just 10 days, aimed at making small animations on web pages (like snow falling on the screen). Nobody thought it would be used for massive systems.
Then came the era of massive web applications (Web 2.0). Microsoft was trying to build the **Office 365** suite directly in the browser. They had millions of lines of JavaScript code. Because of JS flexibility (dynamic typing), one team would change a variable from Number to Text, and another team across the world had a mathematical function relying on that variable. The result? The system crashed completely.

To save the situation, in **2012**, Microsoft (led by **Anders Hejlsberg**, the genius behind the C# language) released TypeScript. The goal was not to destroy JavaScript, but to bring the rigor and discipline (Static Typing) of Enterprise languages (like Java or C#) into it, allowing companies to build huge applications without the fear of introducing "silent bugs".

1.4 Dynamic Typing (JS) vs. Static Typing (TS): Practical Examples

The biggest difference between the two is the **exact moment** you find out you made a mistake.

Example 1: Changing data types (Variables)

- **JS (Dynamic):** The computer doesn't complain until it actually runs the code.

```
// JS: No protection
let price = 100;
price = "One hundred"; // I put text instead of a number. JS says "That's
OK!"
let discount = price - 20;
console.log(discount); // At runtime we get NaN (Not a Number)! The test
fails!
```

- **TS (Static):** Errors are flagged directly in your editor (VS Code), *before* you even run the test!

```
// TS: Active protection
let price: number = 100; // We declare clearly: This box accepts ONLY
```

```
numbers!
price = "One hundred";
// ❌ INSTANT ERROR IN VS CODE (Underlined in red):
// "Type 'string' is not assignable to type 'number'."
```

Example 2: Function Parameters

Let's assume you have a function that clicks on the X-th element in a list.

- **JS:** The function is blind.

```
function clickOnElement(index) {
    // If you receive text instead of a number, the internal logic breaks.
}
clickOnElement("two"); // JS will accept this and crash later.
```

- **TS:** We define a strict contract (Signature).

```
function clickOnElement(index: number) { ... }

clickOnElement("two"); // ❌ Error: Argument of type 'string' is not
assignable to parameter of type 'number'.
```

Example 3: Working with Objects (Auto-complete / IntelliSense)

- **JS:** If you have a user object, VS Code doesn't know what it contains. If you try to read user.name but the property was actually called user.fullName, JS will silently return undefined. Your test will have a false positive or fail mysteriously.
- **TS:** Thanks to static typing (we will learn about *Interfaces* in Chapter 2), the editor knows perfectly what the object contains. When you type user., VS Code will open an auto-complete menu featuring only valid properties, saving you hours of searching and debugging.

1.5 Why does Playwright use TypeScript?

If TS requires us to write more code (to declare types), why do big companies and Playwright prefer it?

- **Catching bugs while writing code (Compile Time):** As an automation engineer, you don't want to launch 500 tests in CI/CD, wait 20 minutes, just to find out you wrote usrname instead of username.
- **Easy Maintenance:** When the interface changes and Frontend developers modify the data structure, TS will instantly underline in red absolutely ALL 50 tests in your project that need

updating, without you having to run them blindly to find out.

- **Self-Documenting Code:** When you read a TypeScript function written by a colleague, you know exactly what kind of data it expects and what it returns, without having to read its internal logic.

1.6 💡 Did you know...?

💡

- **TypeScript doesn't exist in production:** Although developers write and test code in TS, before the application is actually launched on the internet, all TS code goes through a compiler (e.g., tsc or Babel) that strips away absolutely all its rules and types, leaving behind a clean, mundane JavaScript file. All the TypeScript "magic" lives only on your laptop (during the Development/Testing phase)!
- **Massive Adoption:** According to annual StackOverflow surveys, TypeScript consistently ranks in the top 3 "Most Loved Programming Languages," and today almost all major libraries (including React) rely on it.
- **NASA caught on:** The frameworks used by NASA for the control panels of the Mars rovers use strict typed variants of JavaScript, precisely to prevent unexpected crashes millions of miles away.

1.7 🤯 Interview Trap Question (Interview Challenge)

🧠 This is a favorite question in interviews for QA Automation Engineer roles:
The Interviewer: *"I notice you wrote your Playwright framework in TypeScript. Tell me, please: if the tests had been written in plain JavaScript, would they have executed slower in the Chrome browser than the ones written in TypeScript? Does TypeScript make test execution faster?"*

Your Engineer Answer:

"No, the execution in the browser takes exactly the same amount of time. Web browsers (like Chrome) cannot understand or run TypeScript. Playwright compiles my TypeScript code into

> *clean JavaScript in the background, right before launching the browser. The advantage of TypeScript is not the execution speed (runtime), but the speed and safety of **writing** the code (compile-time). TS helps me, as an engineer, discover errors directly in the editor, before the browser even gets to execute the test."*

1.8 🤖 Engineer Mindset in the AI Era: The "Hallucination-Free" Context

> 🌓 **QA vs. AI: Why does AI love TypeScript?**
>
> - **Context:** Generative AI models (like ChatGPT, GitHub Copilot) work by predicting the next logical word based on the context you provide.
> - **The JS Problem:** If you ask the AI *"write a test that validates a product"*, in JavaScript it doesn't know what properties the "product" has. It might guess the product has product.title, but your app might actually use product.name. The AI starts to "hallucinate" code that looks correct, but will fail the test.
> - **The TypeScript Advantage:** In TS, you define interfaces strictly (we will learn this in Chapter 3). When you ask the AI to write the test, it reads the TS typing and clearly *"sees"* that your app uses product.name. Thus, the accuracy of code generated by AI assistants on TypeScript projects jumps from 60% to over 95%!
> - **Your Future:** By adopting TypeScript, not only do you write safer code, but you provide your AI robot with the necessary guardrails to help you at maximum capacity, transforming you from a simple tester into an "AI-Assisted Test Architect".

Chapter 2: First lines of code - Primitive types, any, unknown, and void

Learning Objectives

- 🎯 Discover the testing environment (TypeScript Playground) to write TS directly in the browser.
- 🎯 Understand how to add data types to variables using "Type Annotations".
- 🎯 Learn to rely on "Type Inference", letting TypeScript deduce types automatically.
- 🎯 Discover the essential difference between any (the enemy of quality) and unknown (the defensive approach) through live executed examples.
- 🎯 Use the void type to write clean testing functions.

2.1 Where do we run the code? Playwright Magic

vs. Browser Reality

Unlike previous modules, we have a major technical problem: **Web browsers (Chrome, Firefox, Safari) DO NOT know how to read TypeScript.** They only know the core languages: HTML, CSS, and plain JavaScript.

If you try to open the Chrome console (F12) and type let age: number = 30, you will get an instant syntax error because the browser doesn't recognize the : number sign. TypeScript code must always be **compiled** (translated and stripped of types) into a JavaScript file before it can be executed.

🚀 **Why does the industry love Playwright? (The Out-of-the-box Magic)**

Traditionally, to do this translation, developers had to install complicated compilers and manually run terminal commands (like tsc) every time they changed a line of code.

Playwright revolutionized testing because **it does absolutely everything for us in the background!** When you write your first Playwright test in a .ts file and press the "Run" button, the framework will compile your TypeScript code into JavaScript in just a few milliseconds, without asking for any manual configuration from you. You write strictly, Playwright runs quickly!

Until we install Playwright, to do the exercises in this module, we will use the official simulator created by Microsoft that does this compilation visually for us: **TypeScript Playground**.

🤯 ❓ **Set up your workspace:**

- Open a new tab in your browser and go to: **www.typescriptlang.org/play**
- In the left panel, you will write the TypeScript code.
- Watch the magic: in the right panel, you see in real-time how your code is translated into plain JavaScript! All your typing errors will be underlined in red on the left.

2.2 Primitive Type Annotations

The basic syntax of TypeScript involves adding a colon (:) followed by the data type, exactly after the name of the variable or parameter.

```
// Syntax: let variableName: Type = value;
```

```
let username: string = "admin_qa";
let testTimeout: number = 5000;
let isLoggedIn: boolean = true;
```

🧐 ❓ **Test it in the Playground:**

Copy the lines above into the TS Playground. Then, on a new line, try writing: testTimeout = "five seconds";.

You will instantly see how the code is underlined in red. You were protected from a bug before even running the application!

TypeScript Magic: Type Inference

Do we always have to write : string or : number? **No!**

TypeScript is an extremely smart language. If you give a variable a value right when you declare it, TS automatically "guesses" the type and locks it in.

```
// How professionals write (cleaner, shorter)
let targetUrl = "https://platform.com";
// ^ TypeScript saw the text and silently applied ": string" in the
background.

// targetUrl = 100; // If you uncomment this, it throws an error!
```

***Golden Rule**:* Use manual annotation (: type) only when you declare an empty variable that you will fill later, or when defining function parameters.

2.3 The "Bad Boy" of TypeScript: The any Type

When Microsoft built TypeScript on top of JavaScript, they had to leave an "escape hatch" to allow developers to migrate old code more easily. That hatch is called any.

When you use any, you are telling the TypeScript compiler: *"Close your eyes, turn off the error radar, I know what I'm doing!"*. Essentially, you have canceled absolutely all the benefits of the language and returned to the wild west of JavaScript.

🧐 ❓ **Test the Danger in the Playground:**

Paste this code into the TS Playground:

```
let mysteriousVariable: any = "Hello, I am a text";

// We try to use a math method on a text!
let wrongCalculation = mysteriousVariable.toFixed(2);
console.log(wrongCalculation);
```

Notice the problem? TypeScript DOES NOT give you any red errors. Your editor thinks everything is perfect. But if you press the **"Run"** button in the Playground (which executes the compiled code in the browser), the console will give you a fatal crash: TypeError: mysteriousVariable.toFixed is not a function. any lied to you and hid the bug!

In top companies, the rule in Code Review is strict: *No any allowed in the project!*

2.4 The Smart Alternative: The unknown Type

If we are not allowed to use any, what do we do when we make an API request with fetch and we effectively *do not know* beforehand what data format the server will return to us? This is where **unknown** comes in.

When a variable is unknown, TypeScript lets you put anything in it, but **it DOES NOT let you use it** until you verify (validate) exactly what it is!

🤓 ❓ **Test the Safety in the Playground:**

Paste this code:

```
let apiResponse: unknown = "200 OK";

// 1. Try to use a String method.
// The line below WILL THROW AN ERROR in the editor, because TS protects
you!
// let bigText = apiResponse.toUpperCase();

// 2. The QA Approach: Type Narrowing
if (typeof apiResponse === "string") {
    // Inside this 'if' block, TS knows 100% for sure that it is a string.
    // Now it allows you to use specific methods, and even gives you auto-
complete!
    let bigText = apiResponse.toUpperCase();
```

```
    console.log(bigText);
}
```

As an automation engineer, unknown is excellent for API testing, forcing you to write defensive and safe assertions before processing the data.

2.5 Functions and the void Type (Nothing)

We talked about data types for variables, but functions also return data. In JavaScript, if a function just clicks a button but has no return keyword, it implicitly returns undefined.

In TypeScript, a function that executes an action but gives nothing back is marked with the **void** (empty) type. This clearly signals to any programmer reading your code that it makes no sense to try and save the result of that function into a variable.

```
// This function returns a Number (calculated)
const calculateSum = (a: number, b: number): number => {
    return a + b;
};

// This function just executes a click, it gives no data back. The type is
: void
const printToConsole = (message: string): void => {
    console.log("The message is: " + message);
    // There is no 'return' here!
};
```

2.6 🧠 Interview Challenge (Technical Question)

> 🧠 **The Interviewer:** "I am looking over your Pull Request. You used any to fetch a response from an API because our server is unstable. Why did I ask you to change it to unknown? What is the technical difference?"
> **Your Engineer Answer:**
>
> "If I use any, I completely disable TypeScript's type checking. The editor will let me execute a math method (like .toFixed()) on that response, even when the server returns a String, which will lead to a fatal runtime crash.
>
> If I use unknown, I maintain architectural safety. TypeScript will force me (at the compiler level)

to write a verification condition (typeof response === "number") *before* I try to access the data. unknown forces me to write resilient tests, while any opens the door for hidden bugs."

2.7 🤖 Engineer Mindset in the AI Era: The "Laziness" of Generative Assistants

🌓 **QA vs. AI: The any Trap in Auto-generation**

- **What AI does well:** AI models understand business logic and write E2E tests very quickly.
- **The Trap (Where AI fails):** Sometimes, when you ask an assistant (e.g., ChatGPT) to build a helper function for your tests, and your data structure is complex and you haven't described it in detail, the AI "gets lazy". To ensure it outputs code that compiles quickly on the spot, the AI will annotate the function parameters with : any.
- **How a QA engineer thinks:** The engineer rejects code generated with any. They know the AI just introduced a "Trojan horse" into the testing framework's architecture. The engineer will replace any with unknown (forcing the AI to write defensive checks) or they will define a strict interface themselves (we will learn this in the next Chapter). You, as a human, must be the Strictness Gatekeeper against generative tools!

Chapter 3: Advanced Structures - Interfaces and Custom Types

Learning Objectives

- 🎯 Understand the limitations of JavaScript objects and the need to add a strict "contract" to them.
- 🎯 Create Type Aliases and use "Union Types" for variables with multiple allowed options.
- 🎯 Define **Interfaces**, the industry gold standard for mapping API responses.
- 🎯 Use optional properties (?) and strict constants (readonly).
- 🎯 Correlate Interfaces with validation scenarios (Assertions) in QA automation.

3.1 The Problem: Chaotic JavaScript Objects

In JavaScript, an Object is just a collection of properties. You can create an object, add new properties on the fly, delete others, or refer to properties that don't even exist (which will silently return undefined).

```
// This is what classic JS looks like: No guarantees
let newTask = {
    title: "Write the Login test",
    completed: false
```

```
};

// Later in the code, someone makes a typo:
console.log(newTask.titel); // Prints undefined (should have been 'title')
```

When you test an API that returns a list of 100 users, you cannot let your automation code guess what that user looks like. You need a contract. This is where Custom Types and Interfaces come in.

3.2 Custom Types (Type Aliases) and Union Types

To create a custom type, we use the **type** keyword. It's like inventing a new word in the TypeScript dictionary, which you can use exactly the same way you use string or number.

One of the most powerful tools for QA is the **Union Type**, marked by the vertical bar | (which means OR).

🧐 ❓ **Test it in the TS Playground (www.typescriptlang.org/play):**

```
// We create a custom type that ONLY accepts 3 specific states
type TestState = "PASSED" | "FAILED" | "SKIPPED";

let myResult: TestState;

myResult = "PASSED"; // ☑ Perfectly valid!
// myResult = "PENDING"; // ❌ ERROR: Type '"PENDING"' is not assignable to
type 'TestState'.

// Very useful for IDs (some APIs return the ID as a number: 12, others as
text: "12")
type TestId = string | number;
let currentId: TestId = 100;
currentId = "A-100"; // Both are accepted without error!
```

3.3 The Gold Standard for Data: Interfaces

While type is great for simple things or unions, when it comes to **Objects** (like the JSON structures you receive from a server), the industry prefers using the **interface** keyword.

An Interface is a template, a "blueprint," or a contract. It tells the compiler: *"Any object that claims to be of this type MUST have exactly these keys and these value types."*

```
// 1. Define the contract (Interface names are written in PascalCase)
interface User {
    name: string;
    email: string;
    age: number;
    isAdmin: boolean;
}

// 2. Create an object and force it to respect the contract
const employee: User = {
    name: "Alex",
    email: "alex@qa.com",
    age: 30,
    isAdmin: true
    // If you forget a property, or try to add an invented one (e.g.,
address), TS gives an error!
};
```

3.4 Interface Magic: ? (Optional) and readonly

Not all data in a system is always present, and not all data is allowed to be modified.

- **Optional properties (?):** If a certain piece of information might be missing from an API, we add a question mark before the :.
- **Readonly properties (readonly):** For database-generated IDs, which are not allowed to be accidentally overwritten by another programmer.

🤔 ❓ **Test the QA scenario in TS Playground:**

```
interface TestScenario {
    readonly id: number;      // Cannot be modified after creation
    title: string;
    riskLevel?: "High" | "Low"; // The question mark means it can be
completely omitted!
}

const loginTest: TestScenario = {
    id: 1,
    title: "Credential validation",
    // Notice that I omitted 'riskLevel' and TypeScript doesn't penalize
me!
};
```

```
// We try to manipulate the ID (Simulate a code mistake)
// loginTest.id = 2; // ❌ INSTANT ERROR: Cannot assign to 'id' because it is a read-only property.
```

3.5 How do we use Interfaces in Playwright Automation?

When you write an E2E test that intercepts an HTTP response from our "Task Tracker" application (which you built in Session 6), the code will look like this:

```
// Define the blueprint for what we expect from the server
interface TaskTracker {
    id: number;
    title: string;
    completed: boolean;
}

test('Verify fetching tasks', async ({ request }) => {
    // We request the data and FORCE it to be an Array of TaskTracker type
    const response = await request.get('/api/tasks');
    const tasks = await response.json() as TaskTracker[];

    // Now, when you type "tasks[0].", VS Code will auto-complete "title" and "completed"!
    // If developers change the API, your test will instantly show a red error.
    expect(tasks[0].title).toBe("Test login validation");
});
```

3.6 🧠 Interview Challenge (Technical Question)

> 🧠 **The Interviewer:** "In TypeScript, you can define the shape of an object using both type and interface. What is the difference between them, and which do you prefer when documenting an API response?"
> **Your Engineer Answer:**
>
> "The differences have blurred a lot in newer versions of TS, and you can use both to describe objects. However, **I prefer to use interface** for API responses and data models because

interfaces allow for 'Declaration Merging' (if you declare the same interface twice, TS smartly combines them). On the other hand, I use type strictly when I need a 'Union Type' (e.g., string | number) or simple aliases that an interface cannot express directly."

3.7 🤖 Engineer Mindset in the AI Era: "Contract-Based Prompting"

🌓 QA vs. AI: Generating Fake Data (Mock Data)

- **The Problem:** One of the most time-consuming tasks for a QA is creating test data (Mock Data). If you tell an AI *"Generate 5 test products in JSON"*, it will invent a format out of its head, which your application will likely reject.
- **How a QA engineer thinks:** The engineer uses Interfaces as the **perfect Prompt** for the AI. They copy the interface from the application code (interface Product { id: string; price: number; inStock?: boolean }) and feed it into ChatGPT with the message: *"Strictly respect this TypeScript interface and generate an Array with 50 Mock objects for my E2E tests"*.
- **The Result:** The generative assistant will spit out a flawless test file in 3 seconds, with mathematically respected data types (no strings accidentally placed instead of numbers). Interfaces transform the AI from a random text generator into a surgical partner.

Chapter 4: The Command Center - Configuring tsconfig.json

Learning Objectives

- 🎯 Understand why the tsconfig.json file was invented and how it defines a "Project".
- 🎯 Discover the fundamental settings (flags) through **Practical Missions** directly in TS Playground.
- 🎯 Understand why strictNullChecks and noImplicitAny are lifesavers for a QA engineer.
- 🎯 Answer interview questions related to environment configuration.

4.1 Why was tsconfig.json needed? (Historical Context)

In the early days of TypeScript, if you wanted to translate (compile) your .ts files into JavaScript, you had to open the terminal and write a manual command, specifying every single rule.

Imagine a project with dozens of rules and hundreds of files. The command looked something like this:

```
tsc file1.ts file2.ts file3.ts --target ES2015 --module commonjs --strict true --removeComments true --noImplicitAny true...
```

This approach was a maintenance nightmare for a team. If a programmer forgot to write --strict in the terminal, they would compile bug-ridden code.

Microsoft's Solution:

> 💡 They invented the tsconfig.json file. The presence of this file in a folder tells the compiler: *"This is the root of a TypeScript Project. Stop waiting for mile-long terminal commands. Simply read the rules from this configuration file and apply them automatically to the entire folder!"*

4.2 Interactive Lab: How flags change your code

*Open TS Playground. In the top menu, there is a button called **"TS Config"**. We will turn rules on and off there to see the effects in real-time.*

Mission 1: target (The Time Machine)

This setting dictates the "age" of the JavaScript that TS will ultimately generate (in the right panel).

Execution:

- Paste this code into the left panel:

```
const greet = () => console.log("Hello QA!");
greet(); // We call the function to see the result
```

- Open the **TS Config** menu and look for **Target**.
- 🔘 **Set it to ES5 (Old code from 2009):** Look at the right panel (.JS). Your modern arrow function has been forcefully transformed (transpiled) into a classic function:

```
var greet = function () { return console.log("Hello QA!"); }; greet();
```

Press "Run" and you will see **"Hello QA!"** in the console.

- 🔘 **Set it to ESNext (Ultra-modern code):** Look to the right. The code remains exactly as you wrote it (const and =>), because TS knows that modern browsers support this syntax.

Mission 2: noImplicitAny (The Laziness Detector)

When checked, TS refuses to "guess" the any type and forces you to write explicit code.

Execution:

- Paste this code on the left:

```
// I intentionally forgot to give a type to the 'price' parameter
function calculatePrice(price) {
    return price * 1.19;
}
console.log(calculatePrice(100)); // Should print 119
```

- Open **TS Config** and look for **noImplicitAny**.
- 🔘 **Deactivate it (uncheck):** The code compiles perfectly. If you press "Run", it works. But it's dangerous: tomorrow a colleague might call calculatePrice("One hundred") and the app will print NaN (Not a Number).
- 🟢 **Activate it (check):** Instantly, the word price is underlined in red. The editor blocks you: *Parameter 'price' implicitly has an 'any' type.* It won't let you proceed until you fix the code by adding (price: number).

Mission 3: strictNullChecks (The Absolute Savior in QA Automation)

We often look for elements on a page that might not exist. If we don't check for them, the test will crash when running (Runtime Error). This setting prevents exactly that.

Execution:

- Paste this code on the left:

```
// We look for a FAKE button that definitely DOES NOT exist on the TS
Playground page
const deleteBtn = document.querySelector('.fake-button');

// We try to click it directly
deleteBtn.click();
```

- Open **TS Config** and look for **strictNullChecks**.
- 🔘 **Deactivate it:** The editor gives you no error. The code seems "correct". Now press the **"Run"** button. The TS Playground console will display a fatal error: TypeError: Cannot read properties of null (reading 'click'). Your test just failed!
- 🟢 **Activate it:** Right in the editor, before you even press Run, deleteBtn is underlined in red

(*Object is possibly 'null'*). TS forces you to be a good QA and fix the logic before running:

```
const deleteBtn = document.querySelector('.fake-button');
if (deleteBtn) {
    // Only if it exists on the screen, click it. Now it won't crash on
Run!
    deleteBtn.click();
} else {
    console.log("The button was not found on the screen.");
}
```

The Supreme Rule: "strict": true

In real tsconfig.json files, you don't check every single option manually. There is a supreme setting called "strict": true. If you enable it, it automatically turns on (in the background) both noImplicitAny and strictNullChecks, plus 5 other draconian safety rules. This is the gold standard for Playwright.

4.3 Minimalist tsconfig.json example for Playwright

When you install the automation framework, it will generate this file at the root of your project:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "commonjs",
    "strict": true, // This is where all the rules learned above are
activated!
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": [
    "tests/**/*", // Tells the compiler to look only in the tests folder
    "utils/**/*"
  ]
}
```

4.4 💡 Did you know...?

- **TypeScript can help you even in plain JS:** If you have an old project written in classic JavaScript and you don't want to convert it to .ts, you can add a tsconfig.json file at the root. Set "allowJs": true and "checkJs": true, and the VS Code editor will instantly start underlining logic mistakes and potential bugs right in your .js files! It is the fastest way to audit legacy code.

4.5 🤔 Interview Challenge (Technical Question)

The Interviewer: "I took over a testing framework written by a colleague who left. I look in tsconfig.json and see that the strict setting is commented out (disabled). What impact does this have on the framework, and what will most likely happen if I change it to true today, before running the tests?"

Your Engineer Answer:

"If strict is disabled, the framework allows for 'lazy' and dangerous code. The former colleague most likely declared functions without specifying parameter types (relying on implicit any) and manipulated DOM elements without checking if they actually existed on the screen (ignoring the potential for null).

If I change it to true right now, it is highly likely that the project will suddenly 'light up red' in dozens of files directly in the editor. The compiler will block me and force me to go to every function to add missing data types and check every web element with an if block before clicking it. Although it takes time to fix, activating strict: true is essential to eliminate instability (Flaky Tests)."

4.6 🤖 Engineer Mindset in the AI Era

🌓 QA vs. AI: AI and imposed limits

- AI generates code tailored exactly to the environment it thinks it operates in. If you give

a generative assistant a short prompt ("write me a function that clicks a menu"), the AI will write a short version, assuming you don't use strict rules (it will ignore the fact that the menu could be null).

- As an Engineer, you must include the architectural environment in your Prompts: *"Write me a function that accesses the menu. Assume the project runs with **tsconfig strict: true**"*. This simple phrase will force the AI to generate defensive code, including type checks (typeof) and if blocks, saving you from having to fix red errors in VS Code later.

Chapter 5: Advanced Concepts (Bonus) - Type Assertions (Type Casting)

Learning Objectives

- 🎯 Understand the limits of type inference when interacting with the Document Object Model (DOM).
- 🎯 Learn to use the as keyword to force a data type (Type Casting).
- 🎯 Realize the dangers of abusing type assertions ("lying" to the compiler).
- 🎯 Correlate this concept with advanced Playwright methods, such as page.evaluate().

5.1 The Problem: When TypeScript knows less than you do

TypeScript is smart, but it's not a mind reader. When you write code that interacts with the web page, TS looks at the document.querySelector('.my-class') function and knows that it will return a generic **Element** (or null).

The problem? A generic Element does not have a property called .value (that belongs only to text fields) and does not have a .disabled property (that belongs to buttons).

🤔 ❓ **Test the ERROR in TS Playground:**

Paste the code below into the left panel. **Do not press Run!** Just look at the visual warning.

```
// Find an element on the page. TS thinks it is a generic "Element".
const emailInput = document.querySelector('.email-field');

// Try to write some text into it
if (emailInput) {
```

```
    emailInput.value = "test@qa.com";
}
```

If you have the strict: true setting enabled, the word value will turn red!

The TS Error: *Property 'value' does not exist on type 'Element'.* TypeScript refuses to let you access .value because, from its point of view, that element could just as well be a or an <h1> which don't have an editable value!

5.2 The Rescue: The as Keyword (Type Casting)

This is where **Type Assertion** comes in. By using the as keyword, you send a clear message to the compiler: *"I know you think this is just a random Element, but I, as an engineer, have inspected the page and I guarantee you it is an Input element. Treat it as such!"*

🤓 ❓ **Test the EXECUTION in TS Playground:**

Delete the old code and paste this one. For the test to work in the Playground, we will create the input ourselves first! Press **Run** and watch the Console!

```
// 1. (Setup) Temporarily inject an input into the Playground's page so we can find it
const fakeInput = document.createElement('input');
fakeInput.className = 'email-field';
document.body.appendChild(fakeInput);

// 2. (The Test) Find the input and force the type using "as HTMLInputElement"
const foundInput = document.querySelector('.email-field') as HTMLInputElement;

if (foundInput) {
    // Now TS gives you auto-complete, accepts the command, and we can see the result!
    foundInput.value = "test@qa.com";
    console.log("Success! We managed to set the value:", foundInput.value);
}
```

Most used DOM types in QA:

- as HTMLInputElement (for <input>, grants access to .value, .checked)
- as HTMLButtonElement (for <button>, grants access to .disabled)
- as HTMLSelectElement (for <select> Dropdown menus)

5.3 The Hidden Danger: Lying to the Compiler

Type assertion is a double-edged sword. When you use as, **you turn off TypeScript's protection for that line of code**. The compiler will take your word for it.

If you are wrong, the application will crash at runtime, exactly like in the old days of pure JavaScript.

```
// Here we are intentionally lying to the compiler. We select a button, but say it's an Input.
const submitButton = document.querySelector('.btn-submit') as HTMLInputElement;

// TypeScript doesn't throw any errors here because we forced it!
// But when running in the browser, this code will fail silently or have chaotic behavior.
submitButton.value = "Hacked";
```

The Senior's Golden Rule: Use as only when you have an absolutely valid technical reason (like DOM selection) and you are 100% sure of the data type. Do not use as any just to quickly get rid of a red error in the editor!

5.4 Connection to Playwright: Executing code in the Browser

When you use Playwright normally (await page.click()), you don't need as HTMLButtonElement because the framework abstracts the interaction for you.

However, in complex testing scenarios (such as extracting all links from an invisible table or retrieving hidden variables from the console), you will use the **page.evaluate()** command. This command takes a piece of your TypeScript code and injects it directly into the Chrome browser,

running it there.

```
// Real example from an advanced Playwright test
const buttonStatus = await page.evaluate(() => {
    // We are inside the browser here! We must use pure TypeScript.
    const button = document.querySelector('#btn-checkout') as HTMLButtonElement;
    return button.disabled;
});

expect(buttonStatus).toBe(true);
```

Without knowing the as keyword, you won't be able to write complex evaluated scripts, because the VS Code editor will refuse to compile your test.

5.5 🤖 Engineer Mindset in the AI Era: Blind Assertions

> 🌓 **QA vs. AI: The Abuse of Type Casting**
>
> - **The Trap (Where AI fails):** Sometimes, an AI assistant will generate code where it makes forced data conversions to escape strict errors, for example: const data = apiResponse as any as User;. This is an extremely toxic practice called "Double Casting", which completely hides structural mistakes coming from the server.
> - **How a QA engineer thinks:** The engineer outright rejects double assertions or the use of as over inexcusable API responses. They prefer to use data schema validation tools (e.g., the Zod library) or write *Type Guards* (manual verification functions), reserving the as keyword exclusively for safe DOM selections (HTMLElement).

Chapter 6: Session 7 Assignment & Extra Practice (Towards Senior Level)

6.1 Homework Assignment (Session 7 Project) & QualiAdept Cloud Evaluation Platform

🎯 **Assignment Objective:** Refactoring the "Task Tracker" application by adding the first layer of TypeScript static typing. You will learn to secure variables, data models, and functions, eliminating the risks of undefined errors and forbidding the unsafe use of the any type.

🏢 The Business Scenario (The Business Task):

📝 Your team successfully delivered version 1.0 of the "Task Tracker" application in Session 6. Before starting to write automated Playwright tests in Session 9, the Lead Architect and the QA team have decided that **the entire codebase must be migrated to TypeScript** to guarantee Enterprise-level stability.

Your task is to transform the logic script into a secure TypeScript module (app.ts). You will introduce explicit types for all manipulated data (primitives, arrays, union types for statuses) and you will eliminate any use of the dangerous any type.

Execution Note: You will write the code in the app.ts file (which takes over and extends the logic from the old app.js). In index.html, it will be linked via <script src="app.ts"></script> (assuming the platform/bundler does the compilation in the background, or you use TS Playground to get the JS).

📋 Technical Requirements (Acceptance Criteria for the validation platform):

- **Typing Primitives & Configuration Variables:**
  - Explicitly declare primitive types for the application's configuration variables:
    - apiUrl: string (The external API URL)

        - maxTasksLimit: number (the 5 tasks limit)
        - isAppReady: boolean (application state indicator)
- **Union Type for Task Status:**
    - To avoid typos when setting states, define a Type Alias named TaskStatus.
    - It must strictly accept one of three literal values: 'todo' | 'in-progress' | 'completed'.
- **Typed Data Model (TaskItem Type Alias):**
    - Define a Type Alias named TaskItem that strictly describes the structure of a task in memory:
        - id: number;
        - title: string;
        - status: TaskStatus;
        - completed: boolean;
        - priority?: 'low' | 'medium' | 'high'; *(optional property)*
    - Declare the main taskList array explicitly typed as a collection of such objects: TaskItem[] (or Array<TaskItem>).
- **Function Signatures & Zero Any Policy:**
    - **The add task function (processNewTask):**
        - The input parameter taskObj must be typed as TaskItem.
        - The function's return value must explicitly be of type: boolean.
    - **The display/report function (displayTask):**
        - It receives the parameter id: number and explicitly returns void (since it only prints to the console without returning a value).
    - **Strict QA Rule:** The use of the : any type is strictly forbidden. Any appearance of any will cause the automated validation to fail!
- **Typed Asynchronous Function (Fetch API & Promise):**
    - The loadInitialTasks function must explicitly specify its asynchronous return value: Promise<void>.
    - The data converted with await response.json() must be asserted to the correct type using type assertion (as TaskItem[]).

💡 QA Automation Analogy (Why this is vital for Playwright):

In Playwright you will work 100% with TypeScript:

- When you search for an element on the page, the method returns a strictly typed Locator.

- When you call actions (click, fill, goto), they all return a Promise<void>.
- When you create data-driven test scenarios, defining types like TaskItem will prevent you from testing non-existent fields, eliminating human errors before the first test even runs!

6.2 Bonus Challenge (Optional): Absolute safety with unknown

The Scenario: You asserted the server response with as TaskItem[]. Although it's a common practice, in super-strict teams, we are lying to the compiler if the server suddenly changes the data format.

Your Mission: Change the assertion. Save the result from await response.json() into a variable of type unknown. Then, write a condition (Type Guard) that checks if the received data represents an Array. Only if it is true, transfer the data into taskList.

6.3 🔑 Solution Keys (Answers & Explanations)

Don't cheat! Try to write the code yourself before checking the solution. This solution will 100% pass the QualiAdept validation robot.

Homework Solution (app.ts):

```
// 1. Typing Primitives
const apiUrl: string = 'https://jsonplaceholder.typicode.com/todos';
const maxTasksLimit: number = 5;
let isAppReady: boolean = false;

// 2. Union Type
type TaskStatus = 'todo' | 'in-progress' | 'completed';

// 3. Typed Data Model
type TaskItem = {
    id: number;
    title: string;
    status: TaskStatus;
    completed: boolean;
```

```
    priority?: 'low' | 'medium' | 'high'; // Optional property
};

// Declaring the typed collection
let taskList: TaskItem[] = [];

// 4. Function Signatures (Zero Any Policy)
const processNewTask = (taskObj: TaskItem): boolean => {
    if (!taskObj.title) {
        console.error("Error: The task must have a title!");
        return false;
    }

    taskList.push(taskObj);
    console.log(`The task "${taskObj.title}" was added.`);
    return true;
};

const displayTask = (id: number): void => {
    const foundTask = taskList.find(task => task.id === id);
    if (foundTask) {
        console.log(`[${foundTask.status.toUpperCase()}]
${foundTask.title}`);
    } else {
        console.log(`Task with ID ${id} was not found.`);
    }
    // Implicitly returns void
};

// 5. Typed Asynchronous Function
const loadInitialTasks = async (): Promise<void> => {
    try {
        const response = await
fetch(`${apiUrl}?_limit=maxTasksLimit`);if(!response.ok)thrownewError("Erro
rconnectingtotheAPI");//Weassertthetypeofthereceiveddataconstserver Data=awa
itresponse.json()asTaskItem[];//PopulateourlisttaskList=serverData;isAppRea
dy=true;console.log(`{taskList.length} tasks were loaded.`);

    } catch (error) {
        console.error("The system encountered a problem:", error);
    }
};
```

```
// Simulate an execution
loadInitialTasks().then(() => {
    // After the data is loaded, we add a new strictly typed task
    const newTest: TaskItem = {
        id: 999,
        title: "Run the Playwright suite",
        status: "todo",
        completed: false,
        priority: "high"
    };

    processNewTask(newTest);
    displayTask(999);
});
```

🎉 **Congratulations!** This is your first valid TypeScript code. You are architecturally ready to tackle the most advanced concepts in Playwright!

Complete Session 7 - TypeScript Core

> 📈 **The Goal:** Build "muscle memory" for Playwright. All the exercises below are **complete scripts**. They contain type declarations, instantiations, method calls, and printed results (so they can be verified).
>
> 🛠️ **Setup:** Open the **TypeScript Playground** (with strict: true enabled). Copy each code block,

paste it on the left, and press **Run**. Check the "Logs" (Console) panel on the right!

📍 Exercise 1: Functions, Primitives, and Type Inference

Scenario: A validation function that calculates if a test took too long. The function is strictly typed and then **called**.

```
// 1. Define the function with strictly typed parameters and an explicit return
const validateExecutionTime = (testName: string, timeMilliseconds: number, maxLimit: number): boolean => {
    console.log(`[RUNNING] Test "${testName}" took ${timeMilliseconds}ms.`);
    return timeMilliseconds <= maxLimit;
};

// 2. CALL THE METHODS and test the scenarios (Pass and Fail)
const loginTest = validateExecutionTime("Successful Login", 1200, 2000);
const checkoutTest = validateExecutionTime("Card Checkout", 3500, 2000);

// 3. Verify the results (Assertions)
console.log(`Login Result: ${loginTest ? "✅ PASSED" : "❌ FAILED"}`);
console.log(`Checkout Result: ${checkoutTest ? "✅ PASSED" : "❌ FAILED"}`);
```

📍 Exercise 2: Union Types & Literal Types (Restricted Options)

Scenario: Simulate a function that clicks a button, but that button can only have 3 specific states. We don't accept just any string.

```
// 1. Define the allowed types (Only these exact 3 words!)
type ButtonState = "active" | "disabled" | "hidden";

// 2. Utility function for testing
const clickButton = (selector: string, state: ButtonState): void => {
    if (state === "disabled") {
        console.log(`❌ ERROR: Cannot click on ${selector} because it is
disabled!`);
        return;
    }
    if (state === "hidden") {
        console.log(`❌ ERROR: Selector ${selector} is not visible on the
screen!`);
        return;
    }
    console.log(`☑ Successfully clicked on ${selector}.`);
};

// 3. CALL THE METHOD in different contexts
clickButton("#btn-submit", "active");
clickButton("#btn-cancel", "disabled");

// ❌ If you uncomment the line below, TS throws an error before running!
// clickButton("#btn-fake", "nonexistent");
```

📍 Exercise 3: Safety with unknown (Type Guards)

Scenario: We simulate reading a response from a third-party API where we don't have structural guarantees, so we check the type *before* calling methods.

```
// 1. A function that receives unsafe data (e.g., from LocalStorage)
const processMysteriousData = (data: unknown): void => {
    console.log("[PROCESSING] Analyzing received data...");

    // TYPE GUARD: We check for sure if it's text
    if (typeof data === "string") {
        console.log(`It's Text! Formatted version: ${data.toUpperCase()}`);
    }
```

```
    // TYPE GUARD: We check for sure if it's a number
    else if (typeof data === "number") {
        console.log(`It's a Number! Price with VAT: ${data * 1.19}`);
    }
    else {
        console.log("⚠ Unknown format. Ignoring.");
    }
};

// 2. CALL THE FUNCTION testing the compiler's safety
processMysteriousData("secret-token-from-server");
processMysteriousData(250);
processMysteriousData(true); // Will enter the unknown format branch
```

📍 Exercise 4: Interfaces and Arrays (Data Manipulation)

Scenario: We have a contract (interface) for users. We will create a list and a method that filters out admin users.

```
// 1. Define the contract (Interface)
interface TestUser {
    readonly id: number;
    username: string;
    isAdmin: boolean;
}

// 2. Create a database (Typed Array)
const qaDatabase: TestUser[] = [
    { id: 101, username: "john_doe", isAdmin: false },
    { id: 102, username: "super_qa", isAdmin: true },
    { id: 103, username: "dev_lead", isAdmin: true }
];

// 3. Method to extract only admins
const findAdmins = (list: TestUser[]): TestUser[] => {
    return list.filter(user => user.isAdmin === true);
```

```
};

// 4. CALL AND VERIFY
const foundAdmins = findAdmins(qaDatabase);

console.log(`Found ${foundAdmins.length} Admin users.`);
foundAdmins.forEach(admin => {
    console.log(`- Privileged access account: ${admin.username} (ID:
${admin.id})`);
});
```

🎯 Exercise 5: Classes and Encapsulation (Page Object Model)

Scenario: This is the core for Playwright! We create a class that represents a web page. We encapsulate the selectors and expose only the public test methods.

```
// 1. Define the Class
class LoginPage {
    // Private selectors (Protected from the rest of the framework)
    private userSelector = "#input-email";
    private passwordSelector = "#input-password";
    private loginButton = "#btn-submit";

    // Variable that holds the URL
    public url: string;

    constructor(startUrl: string) {
        this.url = startUrl;
        console.log(`[POM] Initialized login page for: ${this.url}`);
    }

    // 2. PUBLIC METHODS (Actions that can be called from the test)
    public typeCredentials(user: string, pass: string): void {
        console.log(`Typing '${user}' into
[this.userSelector]...`);console.log(`Typinghardcodedpasswordinto[{this.pas
swordSelector}]...`);
```

```
    }

    public clickLogin(): void {
        console.log(`Clicking on selector: [${this.loginButton}]`);
    }
}

// 3. EXECUTE THE TEST (Instantiate and call methods sequentially)
const loginPage = new LoginPage("https://app.com/login");

console.log("--- E2E Scenario Starts ---");
loginPage.typeCredentials("admin@qa.com", "Password123!");
loginPage.clickLogin();
console.log("--- Scenario Finished ---");

// ❌ Error: loginPage.userSelector = ".another-class"; (Private property!)
```

📍 Exercise 6: Inheritance (Extends & Super)

Scenario: Every page in your application has a Navbar. Instead of writing the menu methods in each class, we create a BasePage from which we inherit functionalities.

```
// 1. Parent Class (General)
class BasePage {
    constructor(protected pageName: string) {}

    public verifyTitle(): void {
        console.log(`[BasePage] Verifying if the page title contains
'${this.pageName}'`);
    }

    public logout(): void {
        console.log(`[BasePage] Clicked 'Logout' from the main Navbar.`);
    }
}

// 2. Child Class (Specific, extends the Parent)
class DashboardPage extends BasePage {
```

```
    constructor() {
        super("Main Dashboard"); // Calls the constructor from BasePage
    }

    public generateReport(): void {
        console.log("[DashboardPage] Downloading test report...");
    }
}

// 3. CALL THE INHERITED METHODS
const myDashboard = new DashboardPage();

// Method coming for free from the parent!
myDashboard.verifyTitle();

// Method specific to the child!
myDashboard.generateReport();

// Method coming for free from the parent!
myDashboard.logout();
```

📍 Exercise 7: DOM Manipulation and Type Casting (as)

Scenario: Simulate an internal browser test (page.evaluate). We have to force the compiler to know that a found element is actually an HTMLInputElement.

```
// 1. (SETUP): We create the input directly in the Playground's memory so
it can be tested live
const DOM = document.body;
DOM.innerHTML = `<input type="text" id="email-test" value="old@qa.com">
                 <button id="save-btn" disabled>Save</button>`;

// 2. (REAL TEST): Type Casting
// We use "as HTMLInputElement" so we have access to the .value property
const inputField = document.querySelector('#email-test') as
HTMLInputElement;
```

```
// We use "as HTMLButtonElement" so we have access to the .disabled
property
const submitBtn = document.querySelector('#save-btn') as HTMLButtonElement;

// 3. CALL METHODS ON THE DOM AND VERIFY
if (inputField && submitBtn) {
    console.log(`The INITIAL value of the input is: ${inputField.value}`);
    console.log(`Is the save button DISABLED? : ${submitBtn.disabled}`);

    // Action: Write new text and enable the button
    inputField.value = "new@qa.com";
    submitBtn.disabled = false;

    console.log(`The NEW value of the input is: ${inputField.value}`);
    console.log(`Is the save button DISABLED? : ${submitBtn.disabled}`);
}
```

🎉 **This is the actual code you will write day in and day out!** With these examples, we haven't just learned theory; we have *instantiated* classes, *called* polymorphic functions, *passed* arguments respecting strict contracts, and visually *validated* the results of DOM manipulation!